

```
arr = [10, 25, 30, 45, 50]
key = 30

for i in range(len(arr)):
    if arr[i] == key:
        print("Key found at index", i)
        break
else:
    print("Key not found")
```

Key found at index 2

```
def binary_search(arr, low, high, key):
    if low > high:
        return -1

    mid = (low + high) // 2

    if arr[mid] == key:
        return mid
    elif key < arr[mid]:
        return binary_search(arr, low, mid - 1, key)
    else:
        return binary_search(arr, mid + 1, high, key)

arr = [5, 10, 15, 20, 25]
key = 20

result = binary_search(arr, 0, len(arr) - 1, key)

if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")
```

Key found at index 3

```
n = 5
factorial = 1

for i in range(1, n + 1):
    factorial = factorial * i

print("Iterative Factorial:", factorial)
```

Iterative Factorial: 120

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
```

```
n = 5
print("Recursive Factorial:", factorial(n))
```

Recursive Factorial: 120

```
n = 6

a = 0
b = 1

fib = []

for i in range(n):
    fib.append(a)
    a, b = b, a + b

print(fib)

def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

n = 6

for i in range(n):
    print(fibonacci(i), end=" ")
```

```
[0, 1, 1, 2, 3, 5]
0 1 1 2 3 5
```

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result
```

```
arr = [38, 27, 43, 3, 9, 82, 10]
```

```
print(merge_sort(arr))
```

```
[3, 9, 10, 27, 38, 43, 82]
```

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[-1]  
  
    left = []  
    right = []  
  
    for x in arr[:-1]:  
        if x <= pivot:  
            left.append(x)  
        else:  
            right.append(x)  
  
    return quick_sort(left) + [pivot] + quick_sort(right)
```

```
arr = [10, 7, 8, 9, 1, 5]
```

```
print(quick_sort(arr))
```

```
[1, 5, 7, 8, 9, 10]
```

```
def tower_of_hanoi(n, source, auxiliary, destination):  
    if n == 1:  
        print("Move disk 1 from", source, "to", destination)  
        return  
  
    tower_of_hanoi(n - 1, source, destination, auxiliary)  
  
    print("Move disk", n, "from", source, "to", destination)  
  
    tower_of_hanoi(n - 1, auxiliary, source, destination)
```

```
n = 3
```

```
tower_of_hanoi(n, "A", "B", "C")
```

```
Move disk 1 from A to C  
Move disk 2 from A to B  
Move disk 1 from C to B  
Move disk 3 from A to C  
Move disk 1 from B to A  
Move disk 2 from B to C  
Move disk 1 from A to C
```

```
def gcd(a, b):  
    if b == 0:  
        return a
```

```
        return gcd(b, a % b)

a = 48
b = 18

print("GCD:", gcd(a, b))
```

GCD: 6

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

arr = [12, 11, 13, 5, 6]

insertion_sort(arr)

print(arr)
```

[5, 6, 11, 12, 13]

```
def heapify(arr, n, i):
    largest = i

    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]

        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)

    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
```

```
heapify(arr, i, 0)

arr = [4, 10, 3, 5, 1]

heap_sort(arr)

print(arr)
```

```
[1, 3, 4, 5, 10]
```

```
def merge_and_count(left, right):
    i = 0
    j = 0
    count = 0
    result = []

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            count += len(left) - i
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result, count

def count_inversions(arr):
    if len(arr) <= 1:
        return arr, 0

    mid = len(arr) // 2

    left, count_left = count_inversions(arr[:mid])
    right, count_right = count_inversions(arr[mid:])

    merged, count_split = merge_and_count(left, right)

    return merged, count_left + count_right + count_split

arr = [2, 4, 1, 3, 5]

sorted_arr, inversions = count_inversions(arr)

print("Number of inversions:", inversions)
```

```
Number of inversions: 3
```

```
arr = [-2, -3, 4, -1, -2, 1, 5, -3]

current_sum = arr[0]
max_sum = arr[0]
```

```
for i in range(1, len(arr)):
    current_sum = max(arr[i], current_sum + arr[i])
    max_sum = max(max_sum, current_sum)

print("Maximum Subarray Sum:", max_sum)
```

Maximum Subarray Sum: 7

```
x = 2
n = 10

result = 1

for i in range(n):
    result = result * x

print("Iterative Power:", result)
```

Iterative Power: 1024

```
def power(x, n):
    if n == 0:
        return 1

    half = power(x, n // 2)

    if n % 2 == 0:
        return half * half
    else:
        return x * half * half

x = 2
n = 10

print("Recursive Fast Power:", power(x, n))
```

Recursive Fast Power: 1024

```
import math

points = [(1, 2), (4, 5), (7, 8), (3, 1)]

min_distance = float("inf")
closest_pair = None

for i in range(len(points)):
    for j in range(i + 1, len(points)):

        x1, y1 = points[i]
        x2, y2 = points[j]

        distance = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)

        if distance < min_distance:
            min_distance = distance
```

```
closest_pair = (points[i], points[j])
```

```
print("Closest Pair:", closest_pair)  
print("Distance:", min_distance)
```

```
Closest Pair: ((1, 2), (3, 1))  
Distance: 2.23606797749979
```